

Pixel Wars — Audit de Performance Backend

Backend Java 21 / Spring Boot d'un jeu multijoueur de placement de pixels par équipes (façon *r/place*). Audit de performance du chemin critique de l'API : chaque version ultérieure est mesurée avec le même protocole.

Contexte & Hot Path

Pixel Wars oppose des équipes sur un plateau de **120 × 80 = 9 600 cases**. Deux routes concentrent l'essentiel du trafic : le **hot path**, seule cible de l'audit.

Route	Rôle	Nature de la charge
GET /api/pixels	Lecture continue du plateau	Très fréquente (rendu client)
POST /api/pixels	Placement d'un pixel	Écriture concurrente entre joueurs

Environnement de test

CPU	AMD Ryzen 7 7840HS (Zen4, 8 cœurs / 16 threads, jusqu'à 3,8 GHz)
Cache	L1 32 Ko I + 32 Ko D par cœur (spec Zen4) · L2 8 Mo (1 Mo/cœur) · L3 16 Mo partagé
RAM	16 Go hôte — 7,4 Go alloués à Docker Desktop (WSL2)
OS hôte	Windows 11 Famille (64 bits)
Runtime hôte (hyperfine, micro-benchmarks)	Temurin 21.0.11+10
Runtime conteneur (backend)	Temurin 21.0.12+8, Alpine Linux (JRE)

Backend en conteneur Docker (WSL2), pas sur métal nu : CPU hôte exposé au conteneur, RAM plafonnée par WSL2.

Synthèse des performances par version

Mesure macro via **vegeta** (charge HTTP réelle) : **séquentiel** (1 requête à la fois) vs **concurrent** (25 comptes distincts × 3 rafales).

Latence moyenne, en millisecondes (plus bas = meilleur) :

Version	Stratégie	GET séq.	GET conc.	POST séq.	POST conc.
V0	Baseline (état initial)	16.3	48.8	34.1	216.3
V1	Section critique de <code>placePixel()</code> réduite	8.6	35.0	18.5	86.6
V2	I/O PostgreSQL sortie du verrou	7.2	26.0	12.1	55.6

Signal le plus fort : le POST concurrent explose (34 → 216 ms, ×6,3) là où le séquentiel reste bas — signature d'une sérialisation des écritures, confirmée au diagnostic. En V1, il retombe à 86,6 ms (-60 %) ; en V2, à 55,6 ms (-74 % au total).

Diagnostic de la V0

Deux goulots indépendants, chacun confirmé par un outil dédié.

Rapport de benchmark complet : [benchmark_v0.pdf](#) .

1. POST — contention de verrou

`PixelServiceImpl.placePixel()` est `synchronized` sur toute sa durée : chaque thread doit franchir ce **monitor enter** bloquant, quel que soit le nombre de cœurs disponibles. Un seul joueur pose un pixel à la fois — validation, écriture PostgreSQL, combos/captures et diffusion WebSocket, tout sous le même verrou.

Confirmé à deux niveaux indépendants :

- **Charge vegeta** — sous POST concurrent, la latence moyenne grimpe à **~216 ms** (diagramme en dents de scie). Visible dans `benchmark_v0.pdf` , chapitres « POST : séquentiel vs concurrent » et « POST : latence par ordre de complétion ».
- **JDK Flight Recorder + flamegraph** (`async-profiler`, `--lock`) : **75 événements `JavaMonitorEnter`** (un par placement). Le chemin dominant (`PixelServiceImpl → AbstractQueuedSynchronizer$ConditionObject`) occupe tout le graphe : la contention domine, pas le calcul. Visible dans `benchmark_v0.pdf` , chapitre « Flamegraph : contention sur verrous (POST concurrent) ».

2. GET — parcours de toutes les cases & réallocations

`BoardGrid.snapshot()` parcourt les 9 600 cases à chaque appel, même si peu de pixels sont posés : le coût dépend de la taille du plateau, pas du nombre de pixels.

Mesuré isolément (hyperfine, 200 000 appels, 15 runs) : **~13,9 µs/appel** — noyé dans le bruit réseau à l'échelle actuelle, mais réel et croissant avec la taille du plateau. Visible dans `benchmark_v0.pdf`, chapitre « hyperfine : micro-benchmark de BoardGrid.snapshot() ».

Passage à l'échelle. À nombre de pixels posés fixe (183), le même appel rejoué sur des plateaux plus grands (jusqu'à 6000×6000) confirme la dépendance à la surface : ×3 750 de surface (9 600 → 36 000 000 cases) donne ×3 345 de coût (13,9 µs → **47,2 ms**), quasiment le même facteur. Visible dans `benchmark_v0.pdf`, chapitre « hyperfine : passage à l'échelle de BoardGrid.snapshot() ».

V1 — réduction de la section critique de POST

Ce qui a été modifié. `PixelServiceImpl.placePixel()` n'est plus `synchronized` sur toute sa longueur : le corps a été scindé. Une nouvelle méthode privée `applyPlacement(...)`, **seule** encore `synchronized`, garde l'état partagé entre joueurs — écriture des cases (`BoardGrid` + `LeaderboardTracker`) et combo/capture. Le reste (sauvegarde du joueur, statistiques, achievements, WebSocket) s'exécute hors verrou : ces opérations ne concernent que le joueur qui vient de jouer.

Bénéfice mesuré (via vegeta) : `POST` concurrent passe de 216,3 ms à 86,6 ms (**-60 %**, médiane 204,0 → 81,2 ms). Le motif en dents de scie de la V0 est atténué mais pas supprimé : le verrou restant couvre l'écriture PostgreSQL et le combo/capture, encore partiellement sérialisés.

V2 — sortie de l'I/O PostgreSQL hors du verrou

Ce qui a été modifié. `applyPlacement(...)` ne fait plus le lien avec PostgreSQL. `writeCell(...)` (lecture `findById` + `save` par cellule) est remplacé par deux méthodes : `applyCellMutation(...)`, appelée sous verrou, qui ne fait que lire/écrire `BoardGrid` en mémoire (`previousTeam` récupéré gratuitement avant l'écrasement) ; `persistCell(...)`, appelée après la libération du verrou dans `placePixel()`, qui fait la lecture/écriture PostgreSQL et met à jour `LeaderboardTracker` (structures thread-safe : `ConcurrentHashMap`, `AtomicInteger`). Le verrou ne protège plus que ce qui doit rester une vue cohérente du plateau : la mutation `BoardGrid` et l'évaluation combo/capture, qui la lisent.

Bénéfice mesuré (via vegeta, 3 répétitions) : `POST` concurrent passe de 86,6 ms à **55,6 ms** (médiane 81,2 → 55,1 ms), soit **-36 %** de plus qu'en V1 et **-74 %** au total depuis la V0. Résultat stable sur les 3 mesures (55,6 / 62,9 / 63,2 ms).

Risque assumé. La mise à jour PostgreSQL et `LeaderboardTracker` se fait maintenant quelques instructions après la mutation `BoardGrid` au lieu d'être atomique avec elle : en cas de crash exactement dans cette fenêtre, l'état diffusé aux clients pourrait devancer l'état persisté. Ce risque existait déjà en V1 pour le reste du traitement (statistiques, achievements, diffusion WebSocket) déplacé hors verrou ; V2 l'étend à la persistance des cellules.

Outillage & reproduction

```
# Stack applicative
docker compose up -d

# Vegeta + hyperfine
make bench

# Micro-benchmark seul
make bench-hyperfine

# Scaling plateau
make bench-hyperfine-scaling

# Charges ciblées
make bench-get-sequential
make bench-get-concurrent
make bench-post-sequential
make bench-post-concurrent
```